Parking System CI/CD

Software Design & Programming

Object-Oriented Programming II

Joseph DiBiasi

University of Denver College of Professional Studies

5/15/2026

Faculty: Nathan Braun, MBA

Director: Cathie Wilson, MS

Dean: Bobbie Kite, MLS

Introduction

Our parking system application has grown from humble beginnings into an application containing several different features spread out over a variety of classes. We are ready to move development of this application from our localized development environment to a Kubernetes cluster using Amazon's Elastic Kubernetes Service (EKS) where we can deploy our application for testing and customer interaction. We want to streamline this process and follow an automated continuous integration and continuous deployment (CI/CD) model as best we can. Our goal is to have an application that is always production ready; we will accomplish this by having a single main branch that is always automatically built, tested, and deployed to a pre-production environment. This branch can then be deployed to production at any time. We will accomplish this by utilizing the GitHub Actions platform to turn a once painful manual process into one that is almost fully automated.

System Setup

GitHub is already a familiar repository location that tracks each commit of our parking system application; we can leverage this into a full CI/CD process using the GitHub Actions platform. GitHub actions are a series of actions that we trigger by pushing up commits in a development branch. These actions will either culminate in the successful deployment of the new code or it being rejected from merging into the main branch if certain conditions are not met. These events will be directly based on what we need to deploy that code. We will build artifacts, run automated tests over them, scan them for vulnerabilities, and even ensure the

code meets an acceptable quality; all of this will be done automatically as developers push up newly committed code to our project repositories. YAML files will let us tweak each of these individual actions to meet our exact needs. Each event will happen in sequence so that we can ensure we are not wasting any actions, after all there is no sense in checking the quality of the code if it cannot be built. The benefit to spending so much time creating automated quality gates for every commit to pass is that we can be confident that any commit successfully able to run that gauntlet can then be deployed straight to our AWS Kubernetes cluster.

Developer Interaction

Automating this entire process almost seems like it takes our developers out of the equation; the reality is that it simply gives them more control over the application. Our developers will be able to review the results of each step in the automation process so they can be confident they are bringing quality code in at all times. This goes beyond just fixing any failed tests; tools such as CodeQL or Trivy can scan the code to alert developers to any quality or security issues. Dependabot can help keep an eye on any dependencies brought into the project and alert if there are security issues found with them. Each tool is a GitHub Action so they can review all of these findings without even leaving GitHub. As our list of GitHub Actions grows, we can even bring in Zizmor, a GitHub Action focused on ensuring quality and security in our action workflows.

We want to be able to react to problems quickly, having the code validation and deployment process automated is a big piece of this, but it also means we need to have all developers merge code directly into the main branch. This might seem counter intuitive, but if everyone is looking

at and maintaining the same code base it means there is little difference between the development and production environments. Truly problematic issues arising in our production environment can be fixed locally, validated by our CI process, and deployed by our CD process all in one day thanks to our confidence that the changes will be limited and validated.

The one catch to this is that we are not committing to the full continuous deployment (CD) process. While we are striving to have a system that only introduces reliably good code into the production environment, we do not want to take the chance of something slipping past our guards straight into production when we are unprepared. An accidental commit on a Friday afternoon would run the risk of mis-charging parking lot users all weekend, a mistake that could take our parking office much longer to sort out! Instead, we will always be production ready by continuously deploying our latest code to a pre-production environment. We will strive for a weekly release cadence on Mondays into our production environment, giving us the week to adapt to any issues that may arise. Since this is a fairly aggressive release cadence and everyone will be merging to the main branch, we will implement feature flagging using AWS AppConfig to ensure that larger changes are not disrupting any actions if they are pushed up to production before they are fully developed.

Outputs and Reporting

In an ideal commit workflow, the developers can simply monitor the stages to make sure the commit passes each action. Newly pushed up code will first be verified that it can compile and build the image necessary to deploy it then automated tests will be run over the code. A failure in one of these steps will be reported within the action, logging there will point a developer in the right direction for figuring out the issue. Assuming these stages pass, the next

step will be our security and quality actions checking over the build artifact. Failures in these stages can be reviewed in the action logs, but it is more advisable to navigate to the repo security section where you can look at the scan results in a more organized fashion. Success in all of these actions causes the commit to be automatically merged into the main branch. There is one more step implemented in this process, GitHub Copilot can be configured to automatically review commits as well. The only reason this is separately mentioned is that the comments the AI will leave are considered non-blocking for auto-merge purposes. We still ask our developers to review these comments as sometimes the provided insights are worth following up.

The successful conclusion of the CI process leads to the final GitHub Action modifying our Kubernetes manifest located on our AWS EKS Cluster. Once the manifest has been updated, Kubernetes will detect the change and commence a rolling restart. This process spins up new pods, one at a time, and once a pod is successfully started it will then terminate one of the existing pods. This continues until all outdated pods have been terminated and replaced with the new version. The process is seamless from a user standpoint as there is always a running instance, vastly increasing the availability of the application despite frequent deployments. While this process is technically only continuous for our pre-production environment, the same rolling restart strategy is used for deploying to our production environment.

Summary

Setting up this automated CI/CD process is not going to be an easy one initially. Even once the initial process is implemented, we will still be challenging our team to produce high

quality code at all times thanks to our weekly releases to production. This is a challenge we are ready to handle. Even if there are issues that reach our production environment, our automated CI/CD process gives us the confidence that deploying a resolution is just a step away.

References

Amazon Web Services. n.d. “What Is Amazon EKS?” *Amazon EKS User Guide*. Accessed May 17, 2026. https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html.

GitHub Docs. n.d. “Quickstart for GitHub Actions.” Accessed May 17, 2026. https://docs.github.com/en/actions/get-started/quickstart.

Kovvuri, Deepak, Bharath Gajendran, and Pawan Shrivastava. 2024. “Simplify Amazon EKS Deployments with GitHub Actions and AWS CodeBuild.” *AWS DevOps & Developer Productivity Blog*, May 5, 2024. https://aws.amazon.com/blogs/devops/simplify-amazon-eks-deployments-with-github-actions-and-aws-codebuild/.

Servile, Valentina. 2024. *Continuous Deployment: Enable Faster Feedback, Safer Releases, and More Reliable Software*. Sebastopol, CA: O’Reilly Media.